

Real Estate Prediction

8INF867 - Fondamentaux à l'apprentissage automatique

RYAN COLLOBERT, MOÏSE TSOMANYA, MATHIS AULAGNIER



Data Sommaire

1 Dataset

Présentation du dataset

2 Data analyse

Prétraitement des données

3 Data sciences

Création des modèles

4 Data web

Interface web

5 Data conclusion

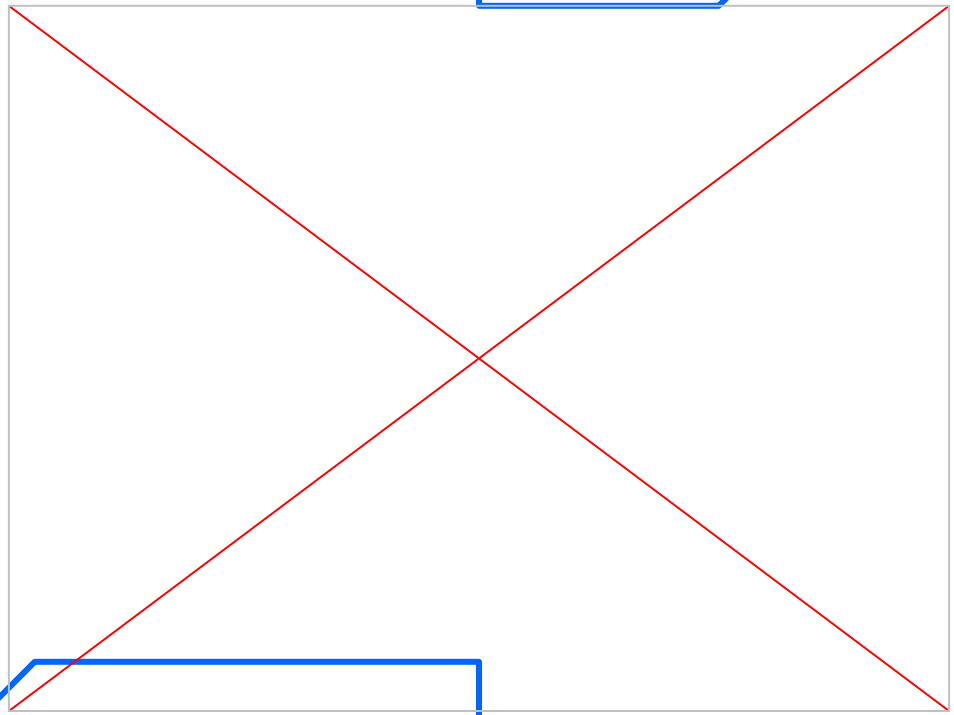


1

Dataset

Housing Prices Competition for Kaggle Learn Users

Ames, Iowa



Ames, Iowa

kaggle



Goal : It is your job to predict the sales price for each house.

Metric : root mean square deviation

Characteristics : 81 columns

Dataset is not cleaned

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\log(\hat{y}_i) - \log(y_i))^2}$$



2

Data analyse

Housing Prices Competition for Kaggle Learn Users

Prétraitement des données

Variables numériques

Traitement des features de type numériques (INT64, FLOAT)

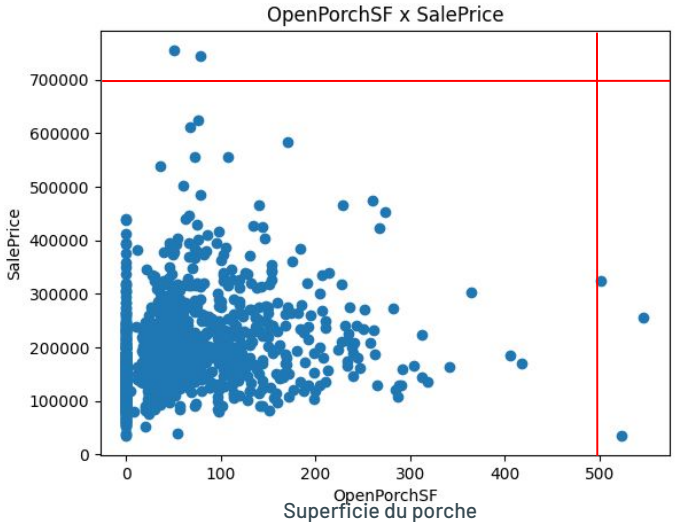
Variables objets

Traitement des features de type objets (STRING, CHAR)

Visuel

Variables numériques

```
for i, classe_name in enumerate(train_df.dtypes[train_df.dtypes != 'object'].index):
    plt.figure(i)
    plt.scatter(train_df[classe_name], train_df['SalePrice'])
    plt.xlabel(classe_name)
    plt.ylabel('SalePrice')
    plt.title(f'{classe_name} x SalePrice')
```



Détection d'anomalies uniquement avec une vision en 2 dimensions.

A???e???????



Variables numériques

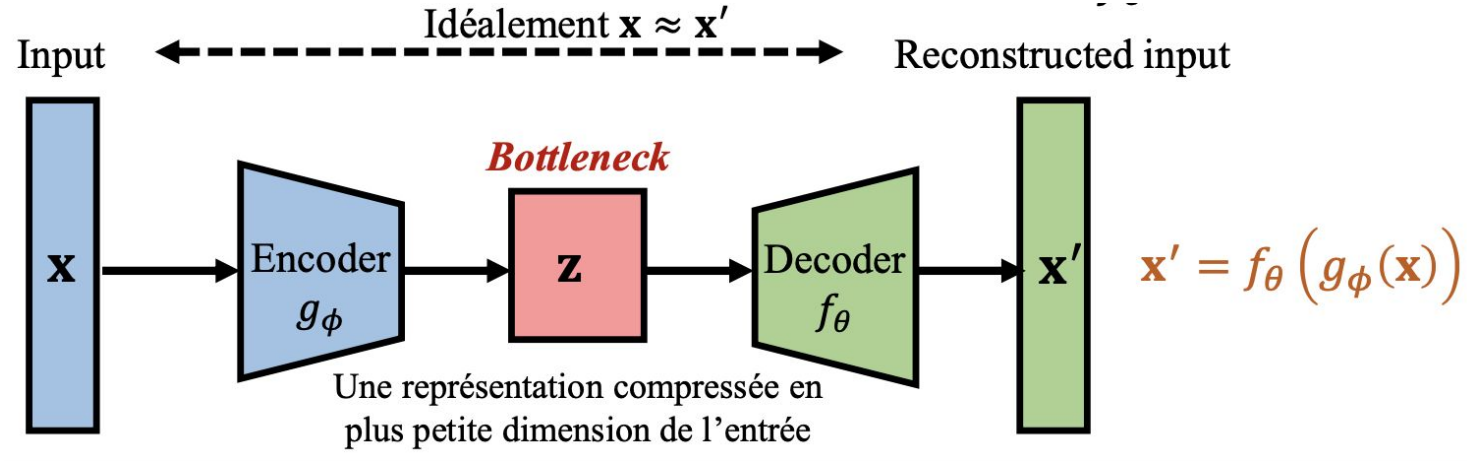
Comment détecter des anomalies dans un espace à 49 dimensions ?



Variables numériques

Comment détecter des anomalies dans un espace à 38 dimensions ?

Les Auto-encodeurs ! (*autoencoders*)



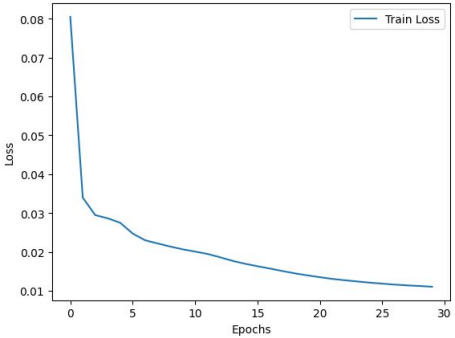
Autoencoders

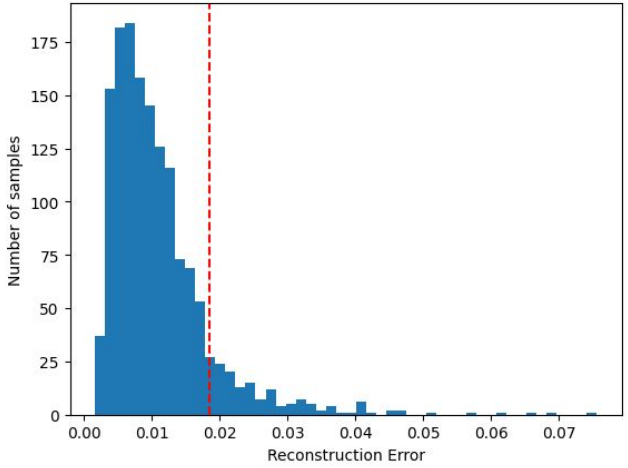


Variables numériques

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	1,248
dense_1 (Dense)	(None, 16)	528
dense_2 (Dense)	(None, 12)	204
dense_3 (Dense)	(None, 16)	208
dense_4 (Dense)	(None, 32)	544
dense_5 (Dense)	(None, 38)	1,254

Total params: 3,986 (15.57 KB)





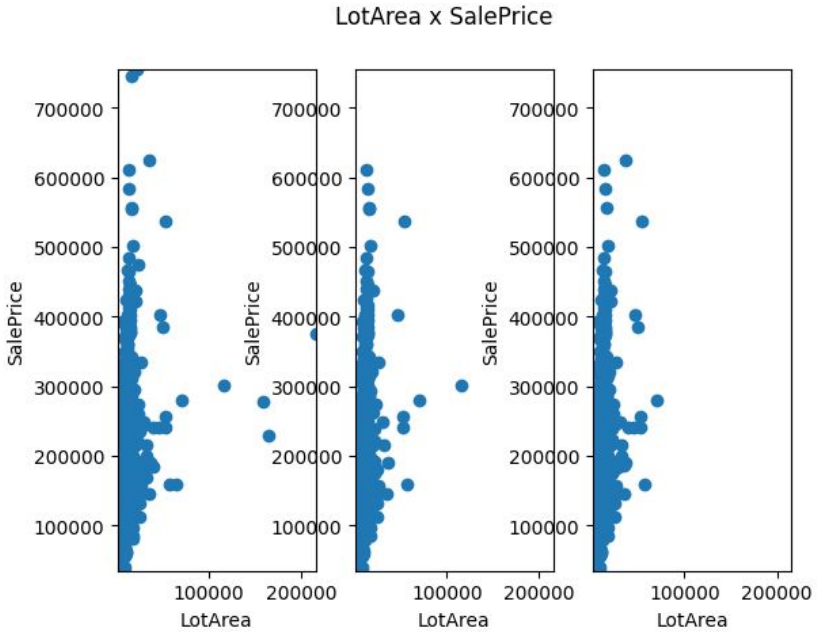
Nombre de valeurs aberrantes détectées : 160

Autoencoders



Variables numériques

Visuel + **Autoencoders**



Original | AE | Visuel



Variables objets

Données manquantes

PoolQC	1296
MiscFeature	1254
Alley	1223
Fence	1050
MasVnrType	758
FireplaceQu	617
LotFrontage	234
GarageYrBlt	50
GarageCond	50
GarageType	50
GarageFinish	50
GarageQual	50
BsmtFinType2	33
BsmtExposure	33
BsmtQual	32

Seuil



Variables objets

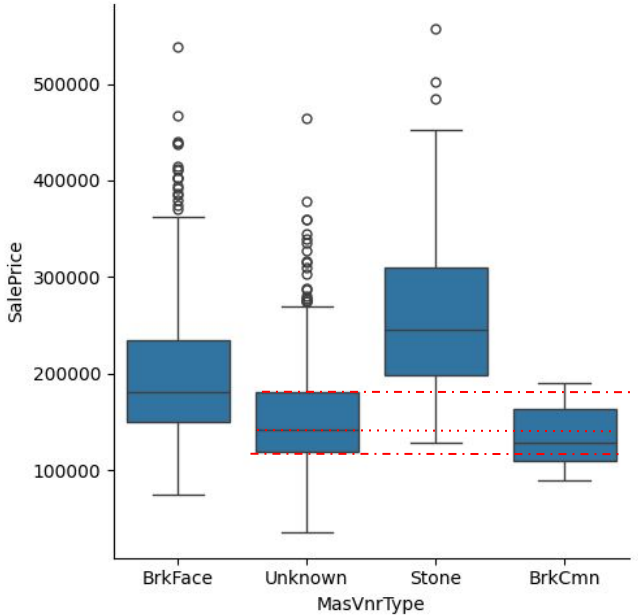
Données manquantes

Seuil

PoolQC	1296
MiscFeature	1254
Alley	1223
Fence	1050
MasVnrType	758
FireplaceQu	617
LotFrontage	234
GarageYrBlt	50
GarageCond	50
GarageType	50
GarageFinish	50
GarageQual	50
BsmtFinType2	33
BsmtExposure	33
BsmtQual	32
BsmtCond	32
BsmtFinType1	32
MasVnrArea	7
Electrical	1
Id	0

+

Associations des inconnues



Variables objets

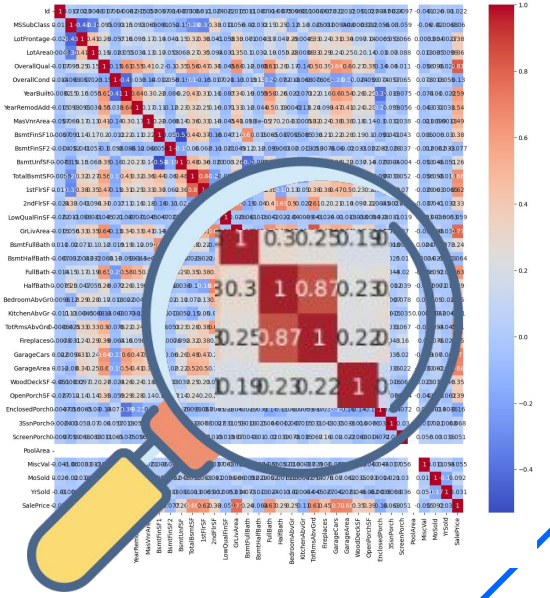
Données manquantes

Seuil

PoolQC	1296
MiscFeature	1254
Alley	1223
Fence	1050
MasVnrType	758
FireplaceQu	617
LotFrontage	234
GarageYrBlt	50
GarageCond	50
GarageType	50
GarageFinish	50
GarageQual	50
BsmtFinType2	33
BsmtExposure	33
BsmtQual	32
BsmtCond	32
BsmtFinType1	32
MasVnrArea	7
Electrical	1
Id	0

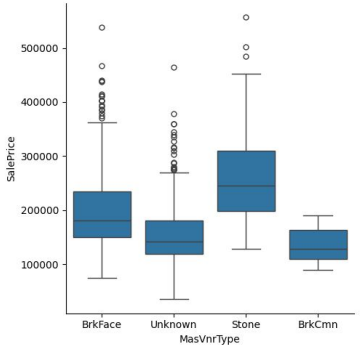
+

Corrélations des variables



+

Associations des inconnues





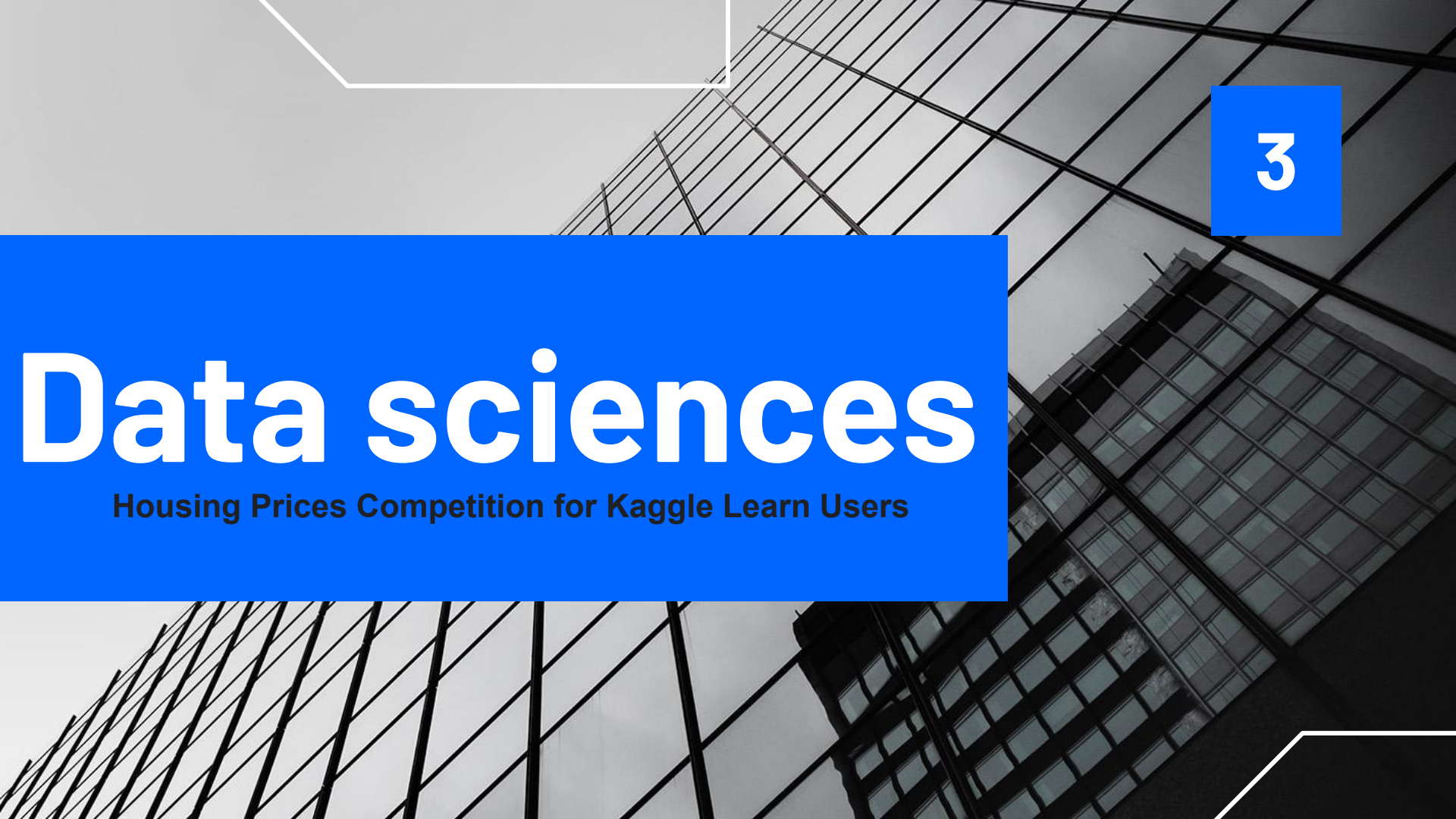
Pipeline de traitement

```python
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')), # Remplace les valeurs manquantes par la moyenne
    ('scaler', StandardScaler()) # Standardise les valeurs (Moyenne = 0, Ecart-type = 1)
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')), # Remplace les valeurs manquantes par 'missing'
    ('onehot', OneHotEncoder(handle_unknown='ignore')) # Encodage one-hot
])

ordinal_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')), # Remplace les valeurs manquantes par 'missing'
    ('ordinal', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)) # Encodage ordinal
])
```

```python
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numerical_columns),
        ('cat', categorical_transformer, categorical_columns),
        ('ord', ordinal_transformer, ordinal_columns)
    ],
    remainder='passthrough', # Ignore les colonnes non transformées
    n_jobs=-1 # Utilise tous les coeurs du CPU
)
```



3

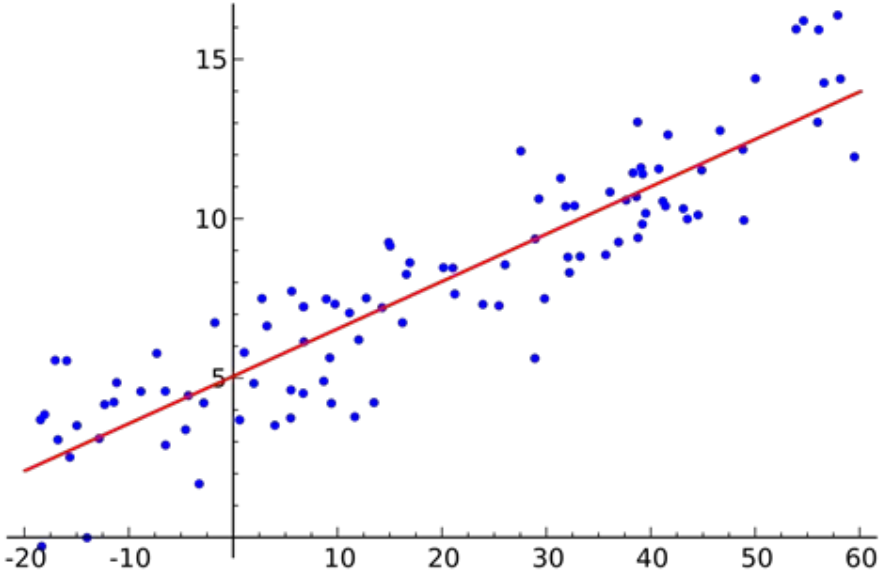
Data sciences

Housing Prices Competition for Kaggle Learn Users

Linear Regression



```
lr = LinearRegression()
lr.fit(X_train, y_train)
```





Ridge

```
ridge = Ridge()
ridge_param_grid = {
    'alpha': [0.05, 0.1, 1, 3, 5],
    'solver': ['auto', 'svd', 'cholesky', 'lsqr', 'sparse_cg', 'sag', 'saga']
}

ridge_grid = GridSearchCV(
    ridge,
    ridge_param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)

ridge_grid.fit(X_train, y_train)
```

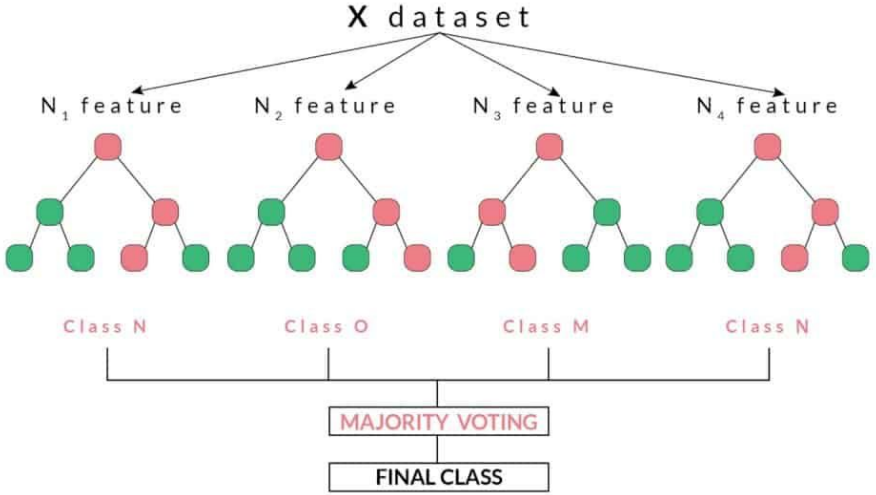
Random Forest Regressor



```
rfr = RandomForestRegressor()

rfr_param_grid = {
    'max_depth': [5, 10, 15],
    'n_estimators': [100, 250, 500],
    'min_samples_split': [3, 5, 10]
}

rfr_grid = GridSearchCV(
    rfr,
    rfr_param_grid,
    cv=5,
    scoring='neg_mean_squared_error' ,
    n_jobs=-1)
rfr_grid.fit(X_train, y_train)
```



Gradient Boosting Regressor

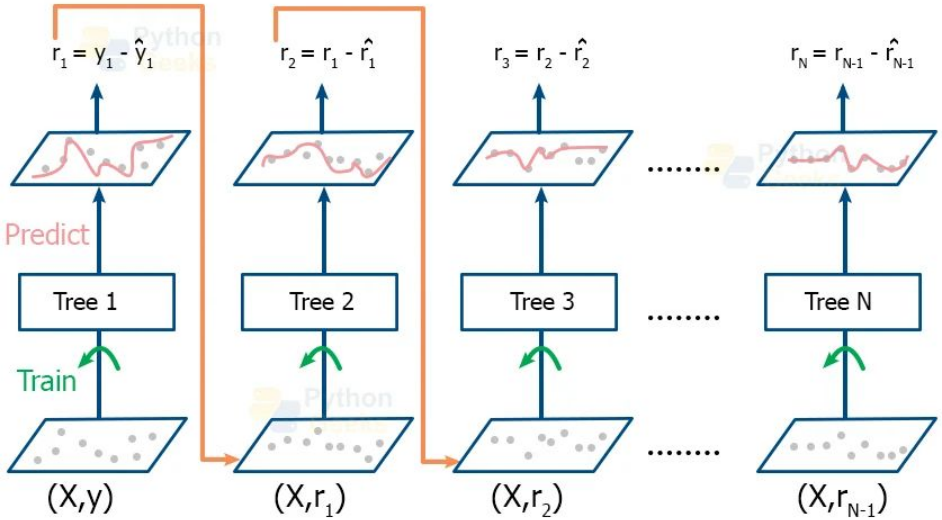
```python
gdr = GradientBoostingRegressor()

gdr_param_grid = {
    'learning_rate': [0.01, 0.1, 0.001],
    'max_depth': [10,15,20],
    'n_estimators': [100, 250, 500, 1000],
    'min_samples_split': [10, 25, 50],
    'max_features': [0.01, 0.1, 0.5]
}

gdr_grid = GridSearchCV(
    gdr,
    gdr_param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)

gdr_grid.fit(X_train, y_train)
```

Working of Gradient Boosting Algorithm

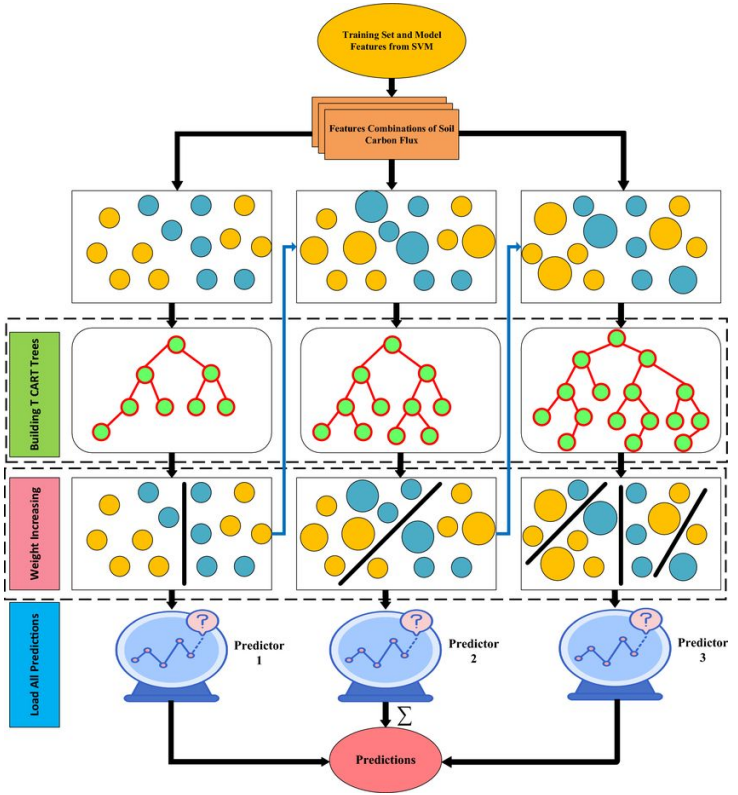


XGB Regressor

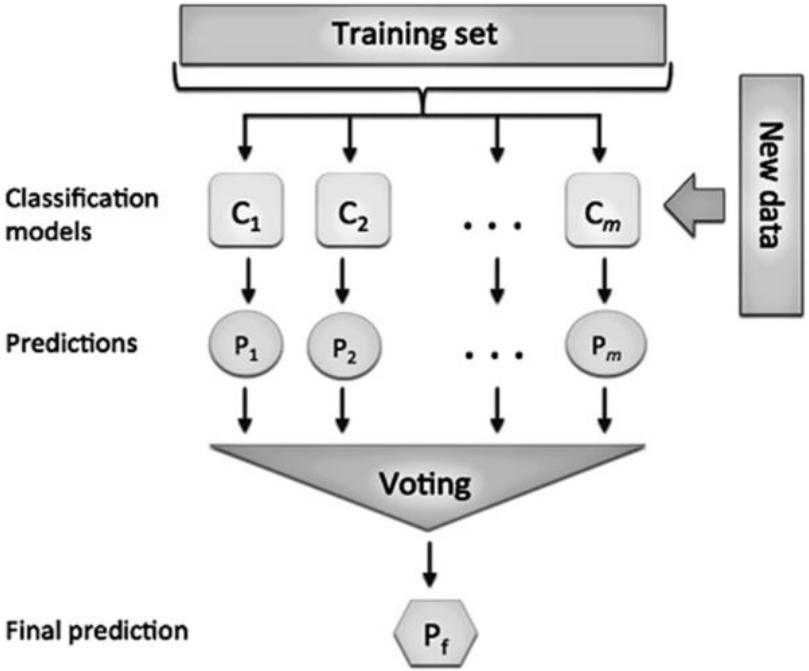
```python
xgb = XGBRegressor()
xgb_param_grid = {
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 5],
    'n_estimators': [100, 250, 500]
}

xgb_grid = GridSearchCV(
    xgb,
    xgb_param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1
)

xgb_grid.fit(X_train, y_train)
```



Voting Regressor

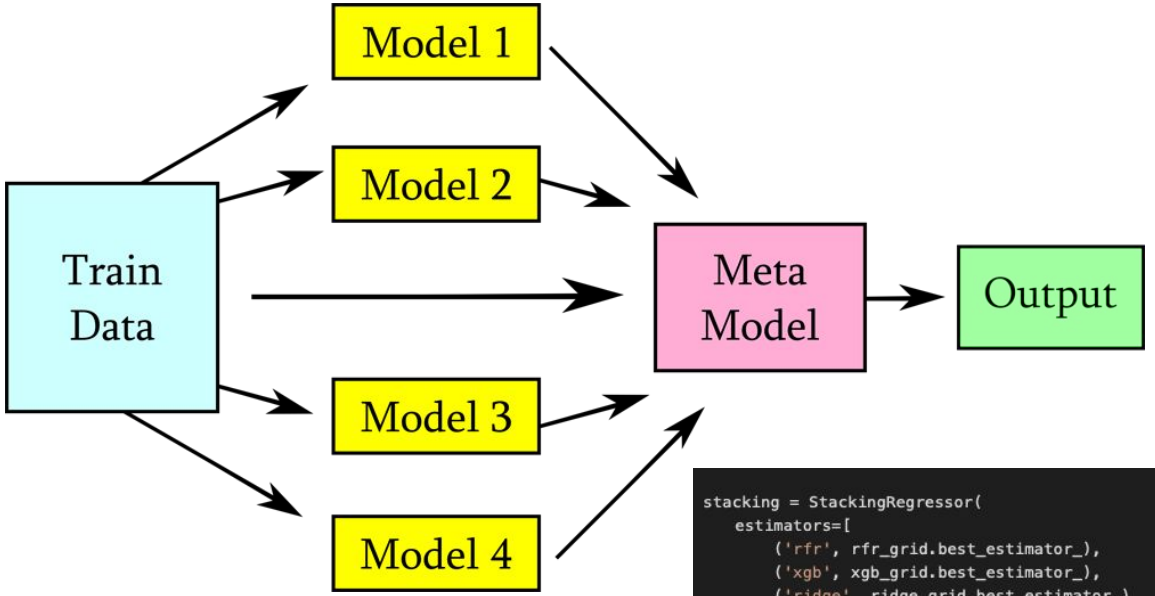



```python
vr = VotingRegressor(
    estimators=[
        ('lr', lr),
        ('rfr', rfr_grid.best_estimator_),
        ('xgb', xgb_grid.best_estimator_),
        ('ridge', ridge_grid.best_estimator_),
        ('gdr', gdr_grid.best_estimator_)
    ],
    weights=[0.2, 0.2, 0.2, 0.2, 0.2]
)

vr.fit(X_preprocessed, y)
```



Stacking Regressor




```
stacking = StackingRegressor(
    estimators=[
        ('rfr', rfr_grid.best_estimator_),
        ('xgb', xgb_grid.best_estimator_),
        ('ridge', ridge_grid.best_estimator_),
        ('gdr', gdr_grid.best_estimator_),
        ('lr', lr)
    ],
    final_estimator=LinearRegression()
)
```



4

Data Web

Housing Prices Competition for Kaggle Learn Users

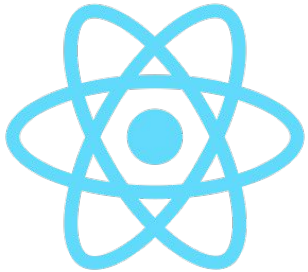


Architecture de l'application

Python Flask pour le back-end

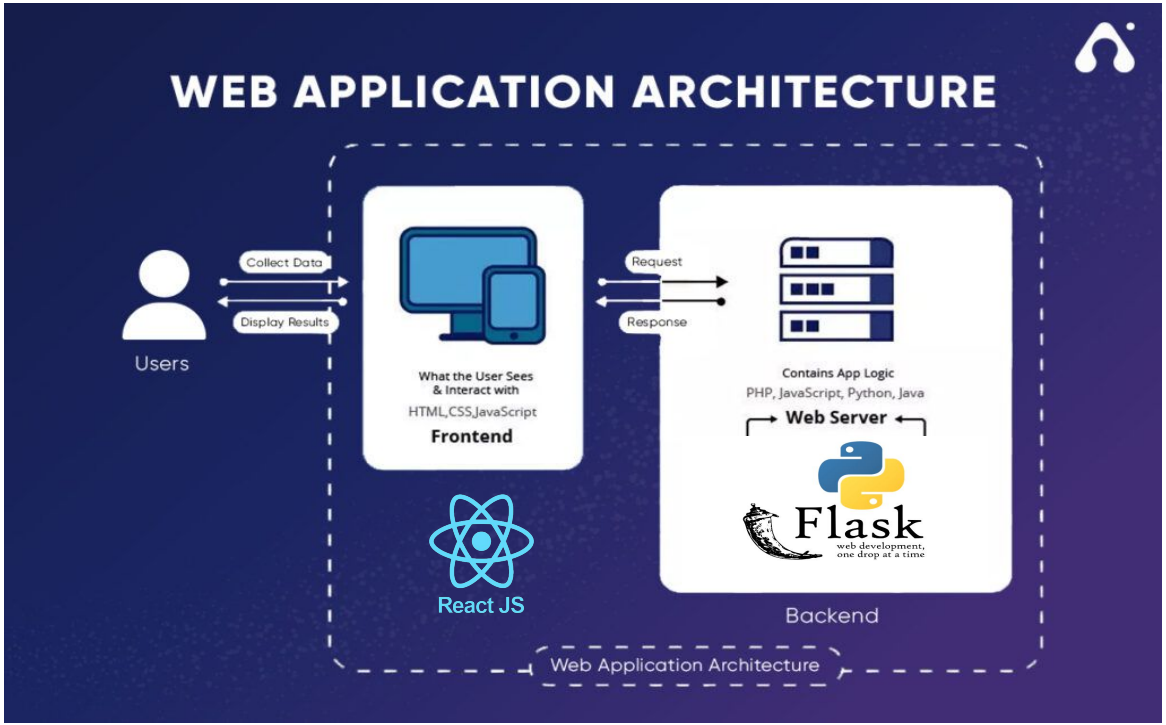


React Js pour le front-end





Architecture de l’application



Architecture de l'application

Requête HTTP

Réponse Serveur

Flask
web development,
one drop at a time

Appel du model

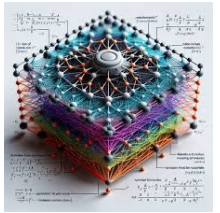
Prédiction du prix

React JS

```python
@app.route('/predict', methods=['POST'])
def predict_house_price():
    data = request.json
    print(data['parameters'])
    try:
        # Validation des données
        if 'parameters' not in data or not isinstance(data['parameters'], list):
            return jsonify({'error': 'Invalid input: "parameters" should be a list'}), 400
        model_path = '../Models/voting_regressor_model.pkl'
        # les données sont elles sous la bonne forme ??!
        instance = create_instance(data['parameters'])
        print("Instance créée avec succès!")

        predictions = predict(model_path, instance)
        print("Prédictions effectuées avec succès!")

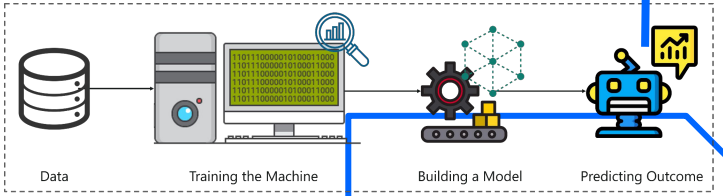
        return jsonify({'prix_predit': predictions[0]})
    except Exception as e:
        return jsonify({'error': str(e)}), 500
```



House price estimation

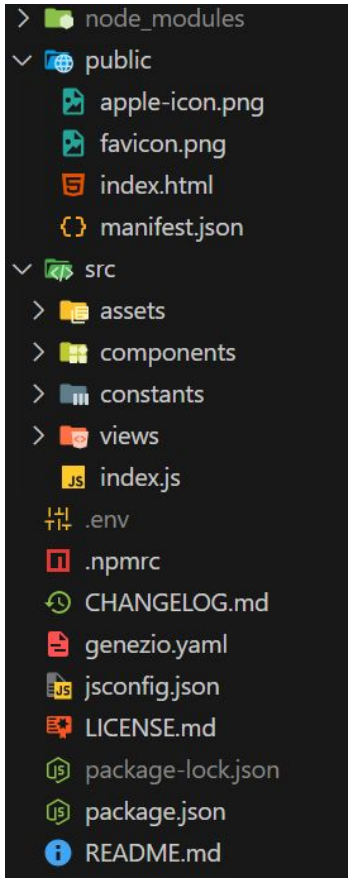
The price of the house is estimated at :

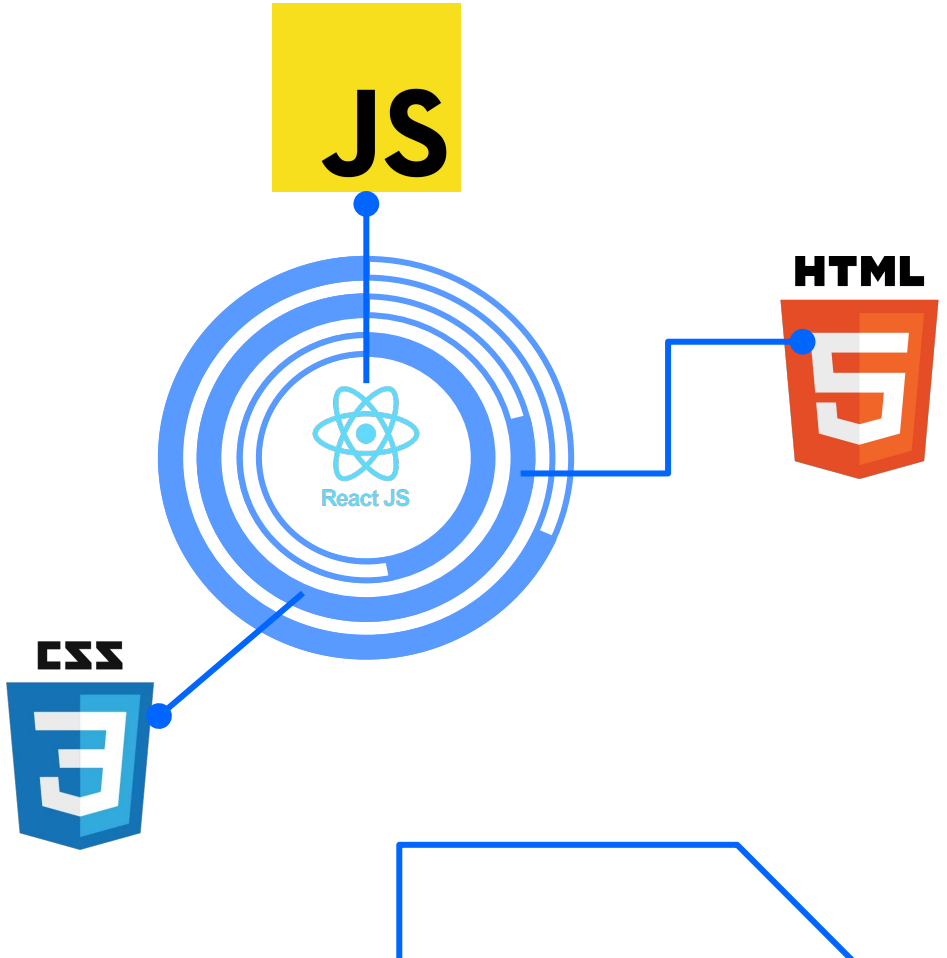
$142,798.43





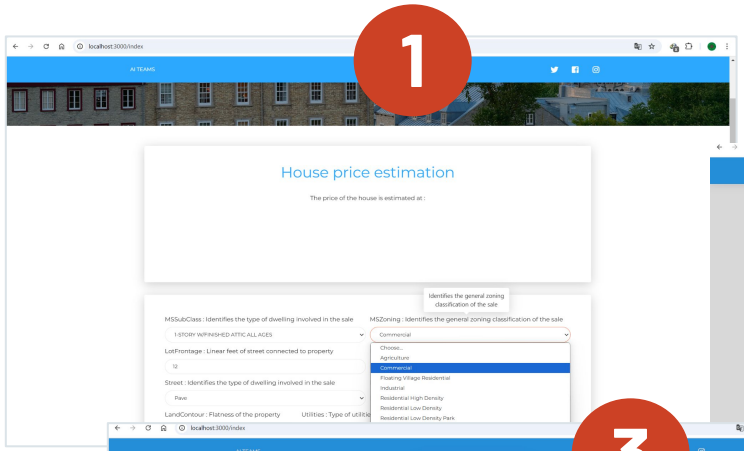
Architecture de l'application

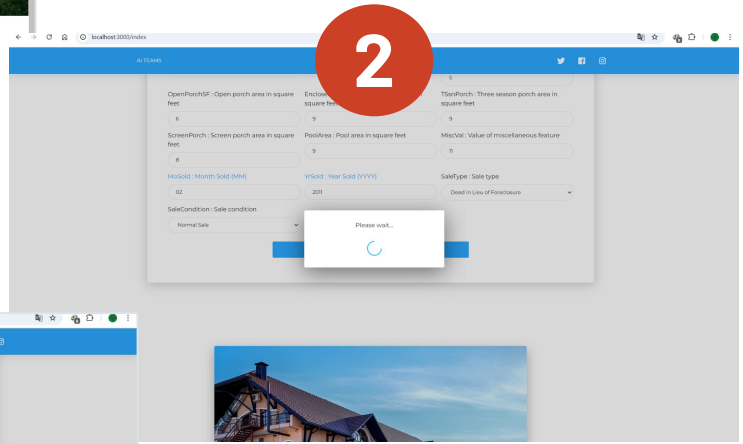


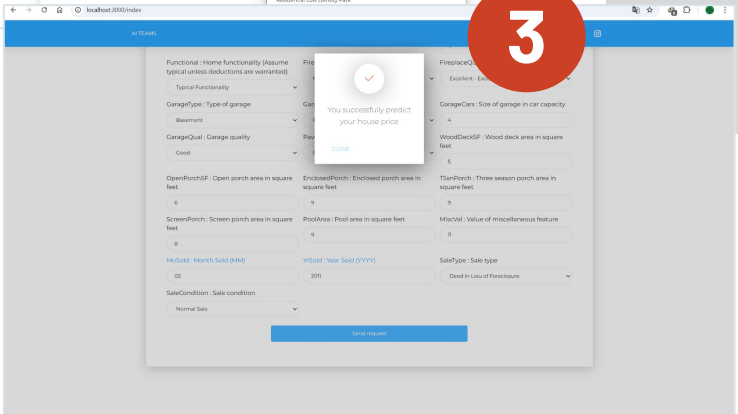




Le front-end


1
localhost:3000/index
AI TEAMS
House price estimation
The price of the house is estimated at :
MSSubClass : Identifies the type of dwelling involved in the sale
1-STORY W/FINISHED ATTIC ALL AGES
MSZoning : Identifies the general zoning classification of the sale
Commercial
Choose...
Agriculture
Commercial
Floating Village Residential
Industrial
Residential High Density
Residential Low Density
Residential Low Density Park
LotFrontage : Linear feet of street connected to property
Street : Identifies the type of dwelling involved in the sale
Pave
LandContour : Flatness of the property

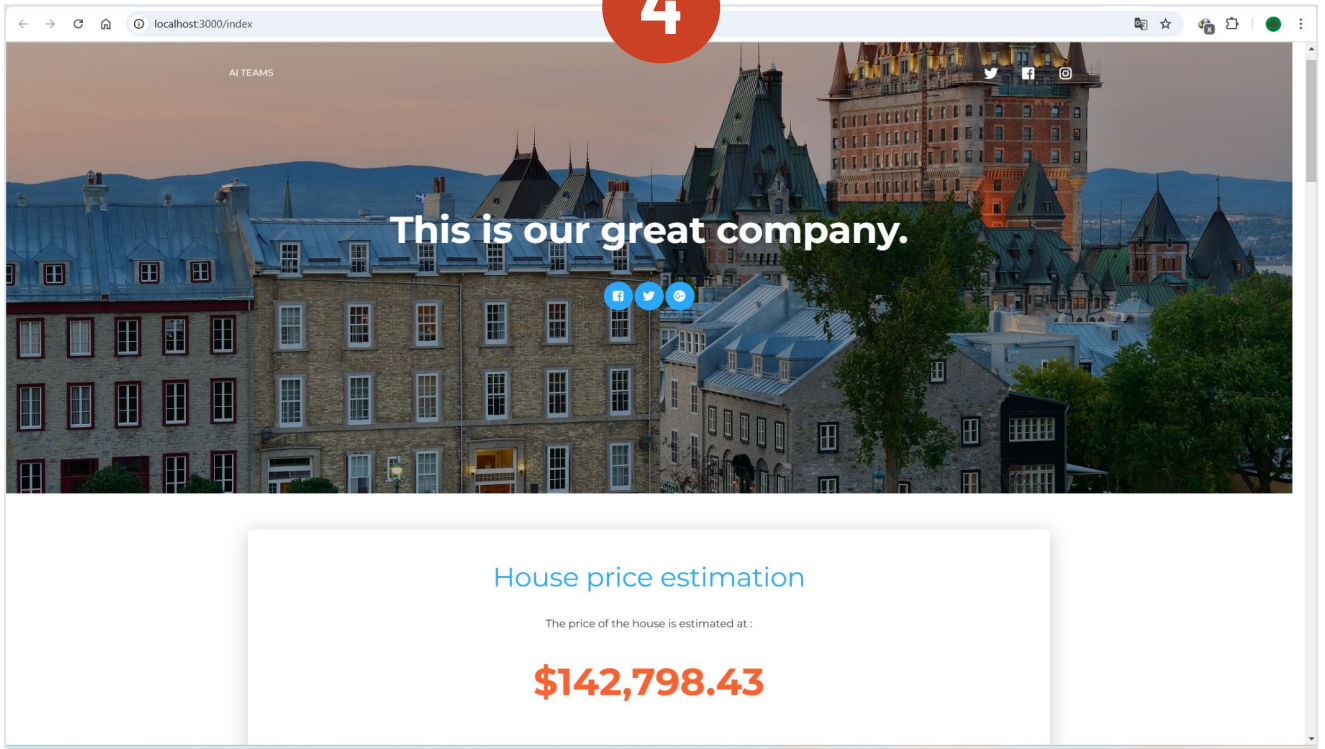

2
localhost:3000/index
AI TEAMS
OpenPorchSF : Open porch area in square feet
TSsnPorch : Three season porch area in square feet
ScreenPorch : Screen porch area in square feet
PoolArea : Pool area in square feet
MiscVal : Value of miscellaneous feature
MoSold : Month Sold (MM)
YrSold : Year Sold (YYYY)
SaleType : Sale type
Deed in Lieu of Foreclosure
SaleCondition : Sale condition
Normal Sale
Please wait...


3
localhost:3000/index
AI TEAMS
Functional : Home functionality (Assume typical unless deductions are warranted)
Typical Functionality
You successfully predict your house price
CLOSE
GarageType : Type of garage
Basement
GarageCars : Size of garage in car capacity
GarageQual : Garage quality
Good
WoodDeckSF : Wood deck area in square feet
OpenPorchSF : Open porch area in square feet
EnclosedPorch : Enclosed porch area in square feet
TSsnPorch : Three season porch area in square feet
ScreenPorch : Screen porch area in square feet
PoolArea : Pool area in square feet
MiscVal : Value of miscellaneous feature
MoSold : Month Sold (MM)
YrSold : Year Sold (YYYY)
SaleType : Sale type
Deed in Lieu of Foreclosure
SaleCondition : Sale condition
Normal Sale
Send request

Le front-end



4





5

Conclusion

Housing Prices Competition for Kaggle Learn Users